Swarm Lab

Assignment 1



BAHRIA UNIVERSITY

H-11 CAMPUS ISLAMABAD

Name: Wisam Saddique
Enrollment: 01-136232-096
Class: BS - AI

DEPARTMENT OF **COMPUTER SCIENCE**
BAHRIA UNIVERSITY | ISLAMABAD H11 CAMPUS

1. Objective

Use Particle Swarm Optimization (PSO) to find a short, obstacle-free path from a start point to a goal point on a 2D grid. The grid, obstacles, start and goal are generated programmatically from my roll number used as the random seed, so my problem instance is unique.

2. My Unique Problem Instance

The roll number is converted to an integer seed by keeping only its digits, then random.seed(seed) is called before anything else is generated. Nothing is hardcoded.

Parameter	Value
Seed	1134231045
Grid size	20x20, obstacles = 88
Obstacle ratio	22% of cells (random.sample)
Start cell (row, col)	(14, 8)
Goal cell (row, col)	(5, 19)
Solvability check	BFS confirms the goal is reachable; otherwise regenerate

3. Approach

3.1 Path representation (what one particle is)

Each particle encodes 6 intermediate waypoints (x, y) in continuous grid coordinates, so a particle is a vector of 12 numbers. The full path is: START -> w1 -> w2 -> ... -> w6 -> GOAL, joined by straight line segments.

3.2 Cost (fitness) function

PSO minimises:

```
cost = path_length  +  100 x (number of collision samples)
```

- **path_length:** sum of the Euclidean lengths of all segments.
- **collisions:** every segment is sampled every 0.05 units. A sample counts as a collision if a small square of half-size 0.15 around it touches an obstacle cell or leaves the grid. This keeps a safety margin and stops the path from slipping between two diagonal obstacles.

The large penalty means any colliding path is always worse than any collision-free path, so the swarm is pushed first towards feasibility and then towards shorter length.

3.3 PSO update equations

```
v(t+1) = w*v(t) + c1*r1*(pBest - x(t)) + c2*r2*(gBest - x(t))
x(t+1) = x(t) + v(t+1)       (clipped inside the grid)
```

The inertia weight w decreases linearly from 0.9 to 0.4 so the swarm explores early and exploits (fine-tunes) later. Velocity is clamped to +/- 20% of the grid size.

3.4 Initialisation

Half of the swarm starts on the straight line from start to goal with Gaussian noise (a good guess that usually needs only small bends), the other half starts uniformly random over the grid (diversity, so the swarm can find detours).

3.5 Parameters

Parameter	Value
Number of particles	60
Waypoints per particle	6
Max iterations	300
Inertia weight w	0.9 -> 0.4 (linear)
c1 (cognitive), c2 (social)	1.5, 1.5
Collision penalty	100.0
Early stopping	no gBest improvement for 60 iterations

3.6 Stopping condition

Stop when 300 iterations are done, or when gBest has not improved for 60 consecutive iterations. gBest is then the final path.

4. How to Run

1. Install the libraries: pip install numpy matplotlib
2. Put your roll number in ROLL_NUMBER at the top of pso_path_planning.py (or pass it on the command line).
3. Run: python pso_path_planning.py (or python pso_path_planning.py 01-134231-045)
4. The program prints the seed, start, goal, path length and waypoints, and saves result_path.png and convergence.png.

5. Results

5.1 Program output

```
Roll number : 01-134231-045
Seed        : 1134231045
Grid        : 20x20, obstacles = 88
Start (r,c) : (14, 8)
Goal  (r,c) : (5, 19)
Iterations  : 300
Path length : 17.443
Collisions  : 0  (obstacle-free)
Final cost  : 17.443
Waypoints (x, y):
  (  8.50,  14.50)
  (  8.87,  12.79)
  (  9.57,  12.77)
  ( 14.44,  13.37)
  ( 16.00,   8.40)
  ( 18.24,   5.88)
  ( 18.77,   5.34)
  ( 19.50,   5.50)
Saved: result_path.png, convergence.png
```

Metric	Value
Best path length	17.443
Collisions	0 (obstacle-free)
Iterations run	300

5.2 Grid, obstacles, start, goal and best path

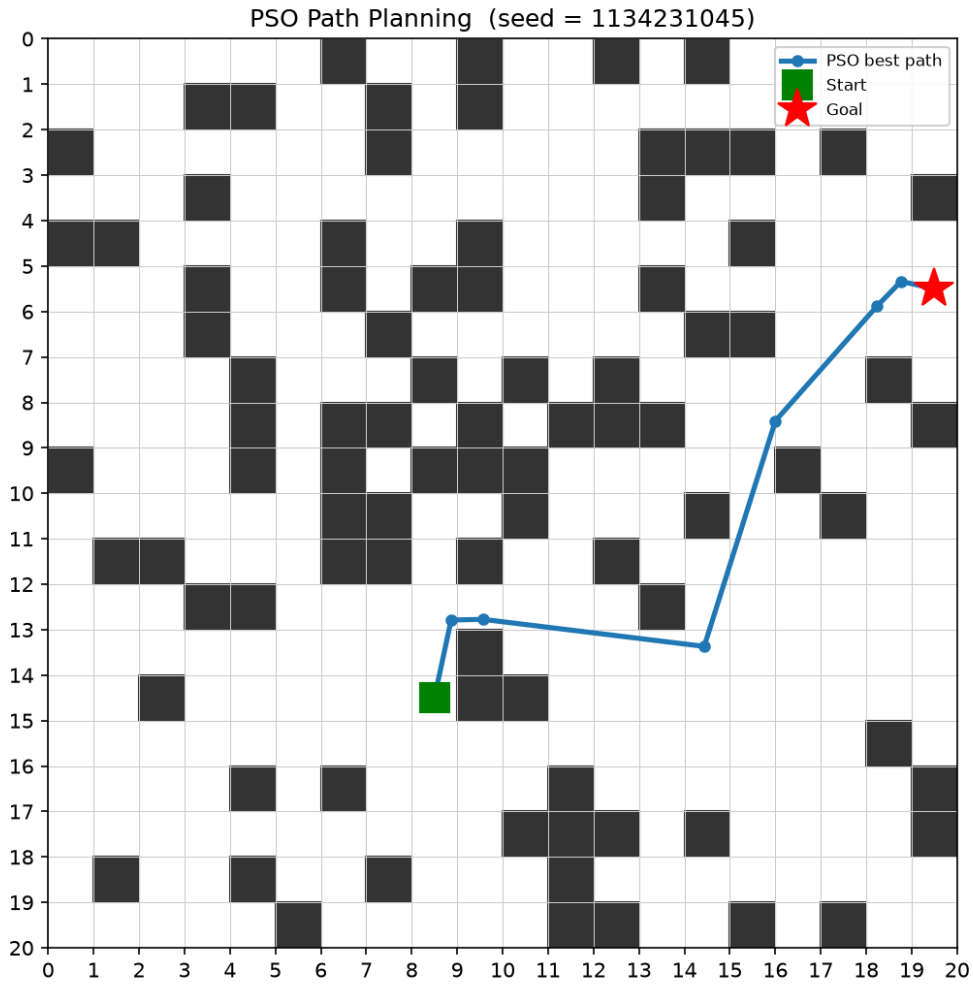
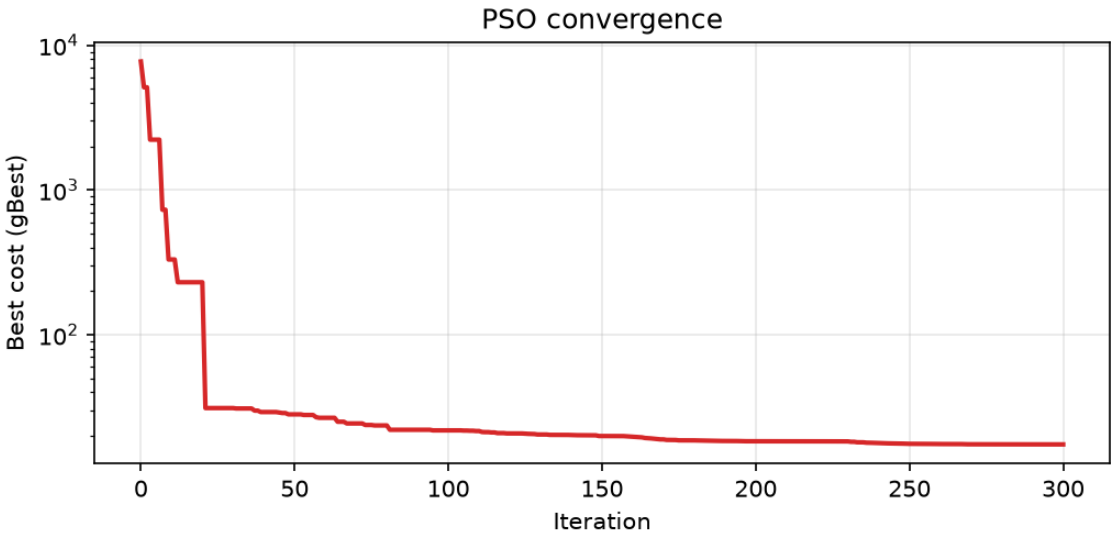
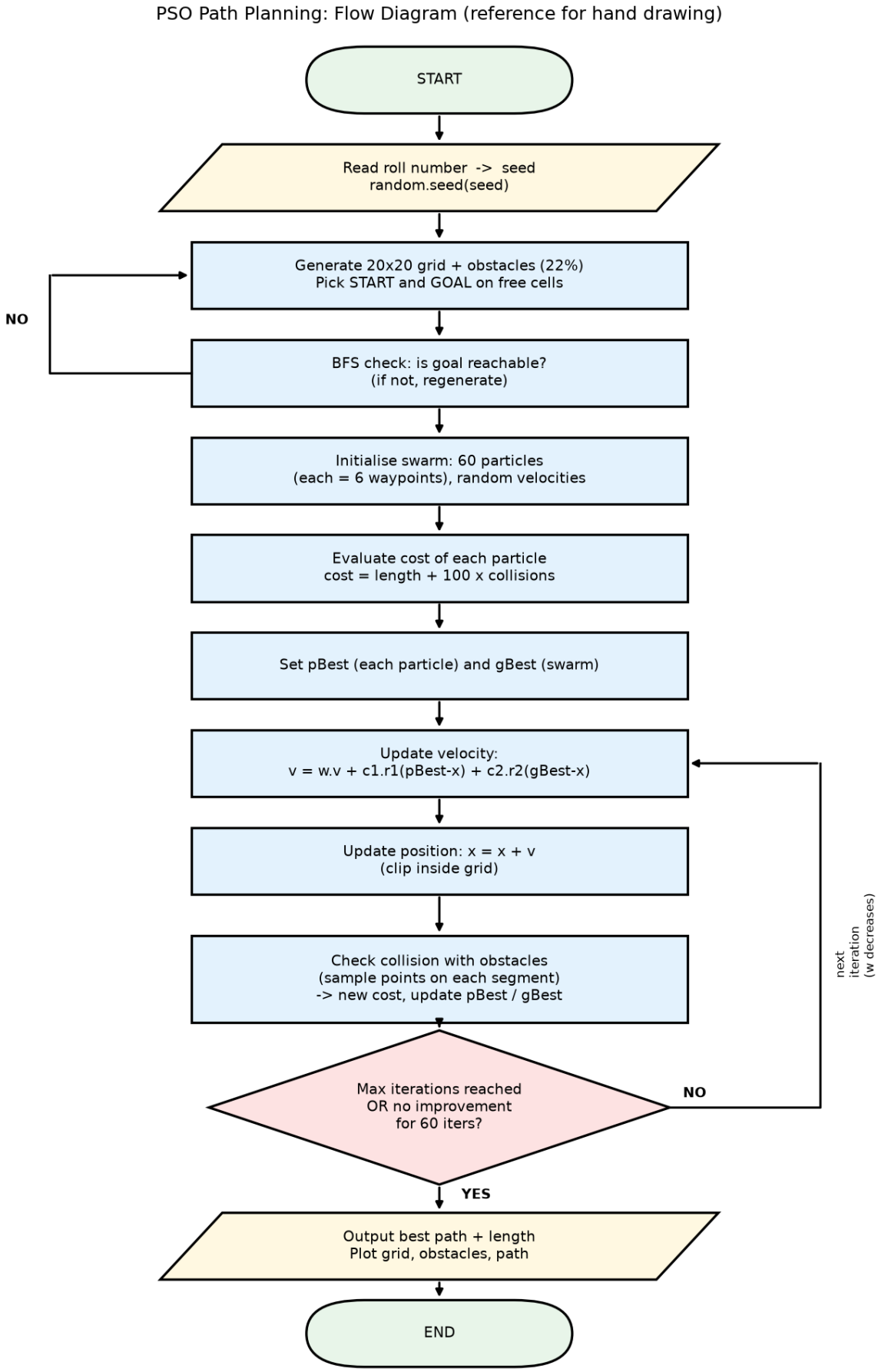


Figure 1: Black = obstacles, green square = start, red star = goal, blue = PSO best path.

5.3 Convergence



6. Flow Diagram



7. Conclusion

PSO found an obstacle-free path of length 17.443 grid units from (14, 8) to (5, 19) on my seeded grid. Encoding the path as waypoints and adding a heavy collision penalty to the length lets standard

PSO handle obstacles without any special operators. PSO is stochastic, so it can settle on a good but not perfectly shortest path; more particles, more waypoints or several runs would improve this.

Appendix: Full Source Code (pso_path_planning.py)

```
"""
Swarm Intelligence Lab - Assignment 1
PSO-based path planning on a 2D grid with obstacles.

Usage:
   python pso_path_planning.py              # uses ROLL_NUMBER below
   python pso_path_planning.py 01-134231-045   # or pass your roll number
"""
import sys
import math
import random
from collections import deque

import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.patches import Rectangle

# ---------------------------------------------------------------
# 1. Student info / seed  (CHANGE THIS TO YOUR OWN ROLL NUMBER)
# ---------------------------------------------------------------
ROLL_NUMBER = "01-134231-045"


def roll_to_seed(roll):
   """Keep only the digits of the roll number and use them as the seed."""
   digits = "".join(ch for ch in str(roll) if ch.isdigit())
   return int(digits)


# ---------------------------------------------------------------
# 2. Problem generation (grid, obstacles, start, goal)
# ---------------------------------------------------------------
GRID_SIZE = 20
OBSTACLE_RATIO = 0.22     # ~22% of cells are blocked
MIN_START_GOAL_DIST = 12  # start and goal should be far apart


def path_exists(grid, start, goal):
   """BFS on free cells to make sure the generated problem is solvable."""
   n = grid.shape[0]
   seen = {start}
   q = deque([start])
   while q:
      r, c = q.popleft()
      if (r, c) == goal:
         return True
      for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
         nr, nc = r + dr, c + dc
         if 0 <= nr < n and 0 <= nc < n and grid[nr, nc] == 0 and (nr, nc) not in seen:
            seen.add((nr, nc))
            q.append((nr, nc))
   return False


def generate_problem(seed, n=GRID_SIZE):
   random.seed(seed)
   while True:  # regenerate (deterministically) until solvable
      grid = np.zeros((n, n), dtype=int)
      all_cells = [(r, c) for r in range(n) for c in range(n)]
      for (r, c) in random.sample(all_cells, int(OBSTACLE_RATIO * n * n)):
         grid[r, c] = 1
      free = [cell for cell in all_cells if grid[cell] == 0]
      start = random.choice(free)
      far = [cell for cell in free
           if math.dist(cell, start) >= MIN_START_GOAL_DIST]
      if not far:
         continue
      goal = random.choice(far)
      if path_exists(grid, start, goal):
         return grid, start, goal


# ---------------------------------------------------------------
```

```
# 3. Path representation and cost function
# ---------------------------------------------------------------------
# A particle = K intermediate waypoints (x, y) in continuous space.
# Full path = start -> w1 -> w2 -> ... -> wK -> goal
# Cell (r, c) occupies the square [c, c+1) x [r, r+1); x = column, y = row.
N_WAYPOINTS = 6
COLLISION_PENALTY = 100.0
SAMPLE_STEP = 0.05
SAFETY = 0.15            # clearance kept from obstacle edges


def cell_center(cell):
    r, c = cell
    return np.array([c + 0.5, r + 0.5])


def build_path(particle, start, goal):
    pts = particle.reshape(-1, 2)
    return np.vstack([cell_center(start), pts, cell_center(goal)])


def path_length(path):
    return float(np.sum(np.linalg.norm(np.diff(path, axis=0), axis=1)))


def collision_count(path, grid):
    """Sample points along every segment and count samples inside obstacles."""
    n = grid.shape[0]
    hits = 0
    for p, q in zip(path[:-1], path[1:]):
        steps = max(2, int(np.linalg.norm(q - p) / SAMPLE_STEP))
        for t in np.linspace(0, 1, steps):
            x, y = p + t * (q - p)
            # check a small square around the point so the path keeps a
            # safety margin and cannot squeeze between diagonal obstacles
            for dx, dy in ((-SAFETY, -SAFETY), (-SAFETY, SAFETY),
                           (SAFETY, -SAFETY), (SAFETY, SAFETY)):
                c, r = int(math.floor(x + dx)), int(math.floor(y + dy))
                if not (0 <= r < n and 0 <= c < n) or grid[r, c] == 1:
                    hits += 1
                    break
    return hits


def cost(particle, grid, start, goal):
    path = build_path(particle, start, goal)
    return path_length(path) + COLLISION_PENALTY * collision_count(path, grid)


# ---------------------------------------------------------------------
# 4. Particle Swarm Optimisation
# ---------------------------------------------------------------------
N_PARTICLES = 60
MAX_ITERS = 300
W_MAX, W_MIN = 0.9, 0.4    # inertia weight (linearly decreasing)
C1, C2 = 1.5, 1.5          # cognitive / social coefficients
PATIENCE = 60              # stop early if gBest doesn't improve


def pso(grid, start, goal, seed):
    rng = np.random.default_rng(seed)
    n = grid.shape[0]
    dim = 2 * N_WAYPOINTS
    lo, hi = SAFETY, n - SAFETY
    v_max = 0.2 * n

    # Initialise: half the swarm on noisy straight lines start->goal, half random
    s, g = cell_center(start), cell_center(goal)
    X = rng.uniform(lo, hi, (N_PARTICLES, dim))
    t = np.linspace(0, 1, N_WAYPOINTS + 2)[1:-1]
    line = np.concatenate([s + ti * (g - s) for ti in t])
    for i in range(N_PARTICLES // 2):
        X[i] = np.clip(line + rng.normal(0, 2.0, dim), lo, hi)
    V = rng.uniform(-1, 1, (N_PARTICLES, dim))

    pbest = X.copy()
```

```
    pbest_cost = np.array([cost(x, grid, start, goal) for x in X])
    gi = int(np.argmin(pbest_cost))
    gbest, gbest_cost = pbest[gi].copy(), pbest_cost[gi]

    history = [gbest_cost]
    no_improve = 0
    for it in range(MAX_ITERS):
        w = W_MAX - (W_MAX - W_MIN) * it / MAX_ITERS
        r1 = rng.random((N_PARTICLES, dim))
        r2 = rng.random((N_PARTICLES, dim))
        # velocity & position update
        V = w * V + C1 * r1 * (pbest - X) + C2 * r2 * (gbest - X)
        V = np.clip(V, -v_max, v_max)
        X = np.clip(X + V, lo, hi)

        # evaluate (cost includes collision penalty) and update bests
        improved = False
        for i in range(N_PARTICLES):
            c = cost(X[i], grid, start, goal)
            if c < pbest_cost[i]:
                pbest[i], pbest_cost[i] = X[i].copy(), c
                if c < gbest_cost - 1e-9:
                    gbest, gbest_cost = X[i].copy(), c
                    improved = True
        history.append(gbest_cost)

        # stopping condition
        no_improve = 0 if improved else no_improve + 1
        if no_improve >= PATIENCE:
            break
    return gbest, gbest_cost, history


# ---------------------------------------------------------------
# 5. Visualisation
# ---------------------------------------------------------------
def plot_result(grid, start, goal, path, seed, fname="result_path.png"):
    n = grid.shape[0]
    fig, ax = plt.subplots(figsize=(7, 7))
    for r in range(n):
        for c in range(n):
            if grid[r, c]:
                ax.add_patch(Rectangle((c, r), 1, 1, color="#333333"))
    ax.plot(path[:, 0], path[:, 1], "-o", color="#1f77b4", lw=2.5, ms=5,
            label="PSO best path")
    ax.plot(*cell_center(start), "s", color="green", ms=14, label="Start")
    ax.plot(*cell_center(goal), "*", color="red", ms=20, label="Goal")
    ax.set_xlim(0, n); ax.set_ylim(0, n)
    ax.set_xticks(range(n + 1)); ax.set_yticks(range(n + 1))
    ax.grid(True, color="#cccccc", lw=0.5)
    ax.set_aspect("equal"); ax.invert_yaxis()
    ax.set_title(f"PSO Path Planning  (seed = {seed})")
    ax.legend(loc="upper right", fontsize=8, framealpha=0.9)
    fig.tight_layout(); fig.savefig(fname, dpi=150); plt.close(fig)


def plot_convergence(history, fname="convergence.png"):
    fig, ax = plt.subplots(figsize=(7, 3.5))
    ax.plot(history, color="#d62728", lw=2)
    ax.set_yscale("log")
    ax.set_xlabel("Iteration"); ax.set_ylabel("Best cost (gBest)")
    ax.set_title("PSO convergence"); ax.grid(True, alpha=0.3)
    fig.tight_layout(); fig.savefig(fname, dpi=150); plt.close(fig)


# ---------------------------------------------------------------
# 6. Main
# ---------------------------------------------------------------
def main():
    roll = sys.argv[1] if len(sys.argv) > 1 else ROLL_NUMBER
    seed = roll_to_seed(roll)
    print(f"Roll number : {roll}")
    print(f"Seed        : {seed}")

    grid, start, goal = generate_problem(seed)
    print(f"Grid        : {GRID_SIZE}x{GRID_SIZE}, obstacles = {int(grid.sum())}")
```

```
    print(f"Start (r,c) : {start}")
    print(f"Goal  (r,c) : {goal}")

    best, best_cost, history = pso(grid, start, goal, seed)
    path = build_path(best, start, goal)
    hits = collision_count(path, grid)

    print(f"Iterations  : {len(history) - 1}")
    print(f"Path length : {path_length(path):.3f}")
    print(f"Collisions  : {hits}  ({'obstacle-free' if hits == 0 else 'NOT free'})")
    print(f"Final cost  : {best_cost:.3f}")
    print("Waypoints (x, y):")
    for p in path:
        print(f"   ({p[0]:6.2f}, {p[1]:6.2f})")

    plot_result(grid, start, goal, path, seed)
    plot_convergence(history)
    print("Saved: result_path.png, convergence.png")


if __name__ == "__main__":
    main()
```